



Fig. 2: LabVIEW front panel

such as ActiveX and TCP/IP networking. LabVIEW lets you create test and measurement, data acquisition, data logging, measurement analysis and report generation applications. It is also possible to create stand-alone executables and shared libraries, like DLLs, because LabVIEW is a true 32-bit compiler. Let us check out some of the important features of this software.

Access to instrumentation hardware

LabVIEW scores over other development environments because it has extensive support for accessing instrumentation hardware. Abstraction layers and drivers for many different types of instruments and buses are included (or are available for inclusion). These present themselves as graphical nodes. The abstraction layers offer a set of standard software interfaces to communicate with the hardware devices. The pre-loaded driver interfaces save program development time. According to NI, the USP of LabVIEW is that even people with limited coding experience can write programs and deploy test solutions in a reduced time frame when compared to working with more conventional or competing systems.

Compiler-produced code

In terms of performance, LabVIEW includes a compiler that produces a native code for the CPU platform. The graphical code is translated into executable machine code by interpreting

the syntax and compilation. The syntax of LabVIEW is strictly enforced during the editing process, and compiled into the executable machine code when requested to run or even while saving. In the latter case, the source code and the executable are merged into a single file. The executable runs with the help of the LabVIEW run-time engine, which contains some pre-compiled code to perform common tasks that are defined by the 'G' language. The run-time engine does the dual function of reducing compile time and also providing a consistent interface to various operating systems, graphic systems, hardware components, etc. The run-time environment is the reason the code is portable across platforms.

Extensive support via libraries and functions

A large number of libraries with many functions for signal generation, data acquisition, mathematics, signal conditioning, statistics, analysis, etc, along with numerous graphical interface elements, are provided in several LabVIEW package options. The fully object-oriented character of LabVIEW code allows for code reuse without modifications as long as the data types of input and output are consistent—which essentially means that two sub VIs are exchangeable.

Create stand-alone executables

The LabVIEW Professional Develop-

ment System allows creation of stand-alone executables, and the resultant executable can be distributed again and again. The run-time engine and its libraries are provided freely along with the executable.

Integrated development environment

The LabVIEW development environment is an application program available for the Windows, Mac and Linux operating systems. It lets users build their own source code, construct graphical user interfaces, compile and run their programs. It also includes libraries for communicating with a wide variety of instrumentation, displaying data in graphical form and performing advanced analysis, making it all the more suitable for scientific and engineering applications compared to general-purpose programming languages.

Multiple execution targets

LabVIEW can be used to develop software that runs on more exotic hardware and not just on a PC. NI has a real-time operating system, which can run on PXI controllers, a PC hardware and cRIO controllers. Using the real-time module on LabVIEW, code can be built to run on these targets, achieving up-time and system determinism not possible under other operating systems. In addition, there are also programmable FPGA targets from NI, in the form of PCI/PXI cards and cRIO backplanes, that allow reconfiguration at the logic circuit level. Through the LabVIEW FPGA module, it is possible to create a solution with the tight timing and low processing overhead of a pure hardware system, but with the easy reconfigurability of a software solution.

Block diagram—graphical source code

LabVIEW replaces text source code with a graphical block diagram, where nodes replace function calls and wires between the nodes representing data transfer. In a manner that's opposite to text-based programming, where functions execute in a sequential manner, in